Arquitetura e Especificação Técnica de Sistemas Agenticos de Recuperação Aumentada por Geração (RAG) em Ambientes Locais

A evolução das arquiteturas de software voltadas para a inteligência artificial generativa atingiu um estágio de maturidade onde a simples interação com modelos de linguagem de grande escala (LLMs) não é mais suficiente para atender às demandas de precisão, privacidade e contextualização de dados privados. O paradigma de Recuperação Aumentada por Geração (RAG) estabeleceu-se como a solução técnica definitiva para mitigar a tendência de alucinação dos modelos estatísticos, fornecendo uma base factual derivada de documentos proprietários.[1] No entanto, a transição de um sistema RAG passivo para um sistema agentico introduz complexidades estruturais que exigem uma engenharia de software rigorosa, focada em estados, fluxos cíclicos e autonomia de decisão.[3]

Este relatório detalha a concepção de um projeto de estudos avançado, projetado para operar inteiramente em infraestrutura local via Docker, garantindo a soberania dos dados e a eliminando a latência e os custos associados a APIs de nuvem.[6] O sistema integra uma interface web moderna, um backend robusto orquestrado por LangChain e LangGraph, e uma camada de dados vetoriais otimizada para recuperação semântica.[4]

Objetivo do Sistema

O objetivo primordial deste projeto é o desenvolvimento de uma plataforma de inteligência documental que permita a ingestão, indexação e consulta inteligente de arquivos locais em um ambiente isolado e seguro.[7] Ao contrário de sistemas de busca tradicionais baseados em palavras-chave, este sistema visa compreender a intenção do usuário e o significado semântico dos documentos, permitindo diálogos complexos onde a IA atua como um pesquisador que sintetiza informações de múltiplas fontes, cita evidências e reconhece lacunas de conhecimento.[1]

A arquitetura proposta busca não apenas responder perguntas, mas também gerenciar o ciclo de vida do documento dentro de um banco de dados vetorial, permitindo que o usuário visualize a origem das informações e interaja com o agente de forma contínua através de uma interface de chat intuitiva.[13] A escolha pela implantação via Docker visa a portabilidade e a reprodutibilidade, permitindo que o ambiente de estudo seja espelhado em diferentes configurações de hardware com esforço de configuração mínimo.[6]

Levantamento de Requisitos

A engenharia de requisitos para sistemas de IA deve equilibrar as expectativas de usabilidade com as restrições computacionais da execução local de modelos de trilhões de parâmetros.[8]

Requisitos de Negócio

Os requisitos de negócio definem o valor estratégico e os objetivos de alto nível que justificam o desenvolvimento da solução em um contexto profissional ou acadêmico.

Requisito de Negócio	Descrição e Impacto	Justificativa Técnica
Soberania e Privacidade de Dados	Garantir que nenhum dado sensível ou documento submetido seja transmitido para servidores externos ou APIs de terceiros.	Conformidade com regulamentações como LGPD e proteção de propriedade intelectual.[7]
Independência de Provedores Cloud	O sistema deve operar funcionalmente sem conexão com a internet, utilizando modelos open-source executados localmente.	Eliminação de custos variáveis de tokens e mitigação de riscos de indisponibilidade de API.[1]
Rastreabilidade e Confiança	Cada afirmação gerada pelo sistema deve ser obrigatoriamente vinculada a um fragmento de documento real através de citações.	Redução da desconfiança do usuário e combate às alucinações inerentes aos LLMs.[1]
Eficiência de Pesquisa Interna	Reduzir o tempo gasto por pesquisadores ou estudantes na localização de informações específicas em grandes volumes de texto.	Aumento da produtividade através da síntese automatizada de múltiplas fontes.[1]

Requisitos Funcionais

Os requisitos funcionais descrevem as ações específicas que o sistema deve realizar para atender aos objetivos do usuário.

1. **Ingestão de Documentos**: O sistema deve suportar o upload de múltiplos formatos de arquivo (PDF, Markdown, Texto plano) e realizar a extração de conteúdo preservando a estrutura de tópicos e metadados.[10]
2. **Processamento de Texto (Pipeline RAG)**: O backend deve automaticamente dividir os documentos em pedaços (chunks) lógicos, gerar representações vetoriais (embeddings) e armazená-los em um banco de dados vetorial.[10]
3. **Chat com Memória de Contexto**: A interface deve permitir uma conversação multi-turno, onde o agente mantém a história do diálogo para responder perguntas de acompanhamento de forma coerente.[3]
4. **Recuperação e Atribuição**: Ao responder, o sistema deve recuperar os fragmentos mais relevantes do banco de dados vetorial e apresentar links ou referências visuais para os documentos de origem.[13]
5. **Gerenciamento de Base de Conhecimento**: O usuário deve ser capaz de visualizar a lista de documentos indexados, remover arquivos individualmente e limpar a base de dados vetorial conforme necessário.[18]
6. **Streaming de Respostas**: O sistema deve suportar a exibição da resposta em tempo real (token por token) para melhorar a experiência do usuário e reduzir a percepção de latência.[3]

Requisitos Não Funcionais

Estes requisitos definem os critérios de operação e as restrições de qualidade do sistema, fundamentais para a estabilidade em execuções locais.

Categoria	Requisito	Detalhamento Técnico
Desempenho	Latência de Recuperação	A busca vetorial em uma base de 1 milhão de documentos deve ocorrer em menos de 100ms.[8]
Escalabilidade	Volume de Dados	O banco de dados vetorial deve ser capaz de gerenciar coleções crescentes sem degradação linear de performance.[25]

Disponibilidade	Orquestração Docker	Todos os serviços (LLM, Vector DB, API, UI) devem ser resilientes e reiniciar automaticamente em caso de falha de container.[6]
Usabilidade	Acessibilidade WCAG	A interface deve seguir as diretrizes WCAG 2.1 nível AA, garantindo contraste adequado e navegabilidade por teclado.[27]
Segurança	Isolamento de Rede	A comunicação entre os containers deve ocorrer em uma rede interna do Docker (bridge), expondo apenas as portas necessárias para o host.[16]
Portabilidade	Especificação de Hardware	O sistema deve ser capaz de rodar em máquinas com hardware variado, suportando fallback para CPU caso uma GPU NVIDIA não esteja disponível.[17]

Arquitetura de Recuperação Aumentada por Geração (RAG)

O sucesso de um sistema RAG não depende apenas do modelo de linguagem, mas da qualidade da recuperação da informação. A arquitetura de dados deve ser projetada para transformar dados não estruturados em conhecimento acionável através de embeddings de alta fidelidade e buscas de vizinhança mais próxima.[7]

Opções de Banco de Dados Vetoriais Recomendadas

Para um ambiente de estudo e execução local via Docker, os bancos de dados vetoriais devem ser avaliados pela eficiência de recursos, facilidade de integração e suporte a metadados.[25]

1. **Qdrant**: Escrito em Rust, o Qdrant é conhecido por sua velocidade extrema e baixo consumo de memória, o que o torna ideal para ambientes locais com recursos limitados.[25] Ele oferece suporte nativo para busca híbrida (vetorial + léxica) e possui uma interface administrativa (dashboard) integrada que facilita a inspeção de vetores e metadados sem

ferramentas adicionais.[6] A recomendação baseia-se na sua maturidade e na facilidade de implantação via imagem oficial do Docker.[10]

2. **ChromaDB**: Frequentemente citado como a escolha preferida para prototipagem e projetos acadêmicos, o ChromaDB destaca-se pela sua filosofia "batteries included" e pela simplicidade da API.[25] Ele pode rodar de forma persistente em um container Docker ou até mesmo em modo embutido, facilitando o desenvolvimento inicial.[9] A recomendação fundamenta-se na sua integração profunda com o ecossistema LangChain, permitindo trocas rápidas de modelos de embedding.[9]
3. **pgvector (PostgreSQL)**: Se o projeto exigir a persistência de dados relacionais complexos além dos vetores, a extensão pgvector para PostgreSQL é a solução mais equilibrada.[31] Ela permite realizar consultas SQL tradicionais e buscas vetoriais em uma única transação, simplificando a arquitetura ao evitar a gestão de múltiplos bancos de dados.[31] A recomendação justifica-se pela robustez do PostgreSQL e pela capacidade de unificar o gerenciamento de documentos e metadados sob um único motor de dados.[33]

Sugestões de Sistemas de Retriever

O "Retriever" é o componente que faz a ponte entre a pergunta do usuário e o banco de dados. Para sistemas profissionais, a busca simples por similaridade de cosseno muitas vezes é insuficiente para capturar a complexidade do contexto.[36]

Sugestão de Retriever	Descrição do Mecanismo	Por que Recomendar
Parent Document Retriever	Este sistema indexa pequenos fragmentos de texto (child chunks) para uma busca precisa, mas recupera o documento maior ou o parágrafo completo (parent chunk) para fornecer contexto ao LLM.[37]	Resolve o dilema entre fragmentos pequenos (bons para busca, ruins para contexto) e fragmentos grandes (bons para contexto, ruins para busca).[39]
Hybrid Search (Dense + Sparse)	Combina a busca vetorial semântica (embeddings densos) com a busca por palavras-chave tradicional (BM25 ou embeddings esparsos).[9]	Garante que o sistema encontre termos técnicos específicos, siglas ou IDs de documentos que podem ser diluídos em um espaço vetorial puramente semântico.[11]

Contextual Compression Retriever	Realiza uma primeira busca ampla e, em seguida, utiliza um modelo menor (reranker) para filtrar e comprimir apenas os fragmentos de texto realmente relevantes para a pergunta antes de enviá-los ao LLM final.[38]	Reduz o ruído no prompt e economiza a janela de contexto do LLM, resultando em respostas mais focadas e custos computacionais menores na geração.[37]

O cálculo da relevância em sistemas puramente vetoriais baseia-se na distância entre vetores no espaço n-dimensional. O uso da distância Euclidiana ou da Similaridade de Cosseno é padrão, sendo a última preferida para processamento de linguagem natural por ser invariante à magnitude dos vetores:

$$\text{Similaridade}(\mathbf{A}, \mathbf{B}) = \frac{\sum_{i-1}^{n} A_i B_i}{\sqrt{\sum_{i-1}^{n} A_i^2} \sqrt{\sum_{i-1}^{n} B_i^2}}$$

Essa formulação permite que o retriever identifique a proximidade temática entre a consulta e o documento, independentemente da extensão do texto indexado.[10]

Backend: Engenharia de Software e Orquestração Agentica

O backend de um sistema agentico moderno deve ser projetado para gerenciar fluxos de trabalho assíncronos, persistência de estados e a integração harmoniosa entre diversos serviços containerizados.[4]

Stack Tecnológica e Dependências

A escolha das bibliotecas e frameworks para o backend reflete o estado da arte do desenvolvimento em Python para IA.

- **FastAPI**: Utilizado como framework web principal devido ao suporte nativo para programação assíncrona, fundamental para lidar com operações de I/O de longa duração, como geração de texto e processamento de documentos.[10]
- **LangChain & LangGraph**: LangChain fornece os componentes modulares para RAG, enquanto o LangGraph permite a criação de grafos de estados complexos, transformando a cadeia linear em um sistema cíclico onde agentes podem refletir e corrigir seus próprios erros.[3]
- **Ollama Python SDK**: Necessário para a comunicação local com o servidor Ollama, permitindo o gerenciamento de modelos e a inferência de embeddings e chat sem sair do ecossistema Python.[6]

- **Pydantic v2**: Garante a integridade dos dados através de validação de tipos rigorosa e serialização eficiente, essencial para definir as interfaces de API e os estados do agente.[7]
- **FastEmbed**: Uma biblioteca leve e rápida da Qdrant para geração de embeddings em CPU, eliminando a necessidade de frameworks pesados como PyTorch ou TensorFlow para a etapa de vetorização inicial.[32]
- **SQLModel**: Uma camada sobre SQLAlchemy e Pydantic que simplifica o gerenciamento de metadados relacionais e histórico de conversas no banco de dados.[24]

Propostas de Implantação e Padrões de Design

A implementação do backend deve seguir padrões que garantam a manutenibilidade e a extensibilidade do sistema, especialmente quando se trata de comportamentos probabilísticos de LLMs.[4]

Proposta 1: Arquitetura Modular com Factory Pattern (Foco em Clean Code)

Nesta abordagem, o sistema é dividido em camadas claras: API, Serviço de Negócio, Repositório de Dados e Provedor de LLM.[45] O uso do **Design Pattern Factory** permite que o desenvolvedor troque facilmente o provedor de modelos ou o banco de dados vetorial sem alterar a lógica principal.

- **Padrão de Design**: Factory e Strategy. O sistema define uma interface abstrata para o "Retriever" e o "Generator". No momento da execução, o Factory instancia a implementação específica (ex: QdrantRetriever ou ChromaRetriever) baseada na configuração.[24]
- **Aplicação em RAG**: Isso permite testar diferentes modelos de embedding (HuggingFace vs Ollama) apenas alterando uma variável de ambiente, promovendo o desacoplamento do código.[4]

Proposta 2: Orquestração Baseada em Grafo de Estados (Foco em Sistemas Agenticos)

Ao utilizar o **LangGraph**, o backend deixa de ser uma sequência de passos e torna-se um grafo onde cada nó representa uma função (LLM, Busca, Ferramenta) e cada aresta define a lógica de transição.[3]

- **Padrão de Design**: State Machine. O estado do sistema é passado entre os nós, contendo o histórico de mensagens e os documentos recuperados. Edições condicionais determinam se o agente precisa buscar mais informações ou se já está pronto para responder.[4]
- **Aplicação em RAG**: Implementa um fluxo de "Auto-RAG", onde o agente avalia se o documento recuperado é realmente útil. Caso contrário, ele reescreve a pergunta e tenta uma nova busca antes de falar com o usuário.[37]

Proposta 3: Padrão ReAct com Tool Injection (Foco em Autonomia)

Baseada no padrão **Reasoning and Acting (ReAct)**, esta proposta transforma o RAG em uma ferramenta que o agente decide usar apenas se necessário.[49] O backend expõe o banco vetorial como uma "Tool" que o LLM pode chamar via function calling.

- **Padrão de Design**: Agent-Tool Observer. O agente observa a necessidade do usuário e escolhe a ferramenta certa no seu arsenal. O backend gerencia o registro dessas ferramentas e a injeção automática de seus docstrings no prompt do sistema.[4]
- **Aplicação em RAG**: Permite que o agente lide com perguntas triviais ("Oi", "Tudo bem?") sem desperdiçar recursos no banco vetorial, ativando a busca documental apenas para consultas complexas.[22]

Frontend: Interface Web Profissional e Acessível

O frontend é o ponto de contato do usuário com a complexidade do agente. Ele deve equilibrar a estética moderna com a funcionalidade técnica de um sistema de pesquisa.[13]

Stack Tecnológica e Dependências

Para um visual profissional e uma experiência fluida, recomenda-se o uso do ecossistema React moderno.

- **Next.js (App Router)**: Framework principal para React que oferece roteamento eficiente, suporte a Server Components para segurança e integração nativa com streaming de dados.[55]
- **shadcn/ui**: Uma coleção de componentes reutilizáveis construídos com **Tailwind CSS** e **Radix UI**. Ao contrário de bibliotecas de componentes tradicionais, o shadcn/ui fornece o código fonte, permitindo customização total e garantindo que o desenvolvedor seja dono da interface.[53]
- **Tailwind CSS**: Framework CSS utilitário para design rápido e responsivo, permitindo a criação de interfaces modernas com consistência visual através de tokens de design.[55]
- **Radix UI Primitives**: Fornece a base de acessibilidade (WAI-ARIA) para componentes complexos como diálogos, menus suspensos e abas, garantindo que a interface seja utilizável por todos.[53]
- **Framer Motion**: Biblioteca de animações para React, essencial para criar transições suaves, efeitos de streaming de mensagens e estados de carregamento "esqueleto" que melhoram a percepção de performance.[29]
- **React Hook Form & Zod**: Para gerenciamento de formulários de upload e configurações de parâmetros do agente com validação rigorosa no lado do cliente.[24]

Propostas de Implantação e Experiência do Usuário (UX)

O design de interfaces para IA exige novos padrões que comuniquem transparência e

permitam a verificação de fatos.[13]

Proposta 1: Layout de Painel de Pesquisa (Research Dashboard)

Inspirado em ferramentas de produtividade e IDEs, este layout foca na visualização simultânea da conversa e das fontes.[13]

- **Estratégia de UX**: Utiliza um layout de duas colunas. A coluna da esquerda contém o chat persistente. A coluna da direita é um painel dinâmico que exibe o PDF original ou o trecho do documento quando o usuário clica em uma citação no chat.[13]
- **Práticas de Acessibilidade**: Implementa navegação por teclado para alternar entre os painéis e utiliza aria-live nas bolhas de chat para que leitores de tela anunciem o progresso da resposta da IA conforme ela é gerada.[27]

Proposta 2: Interface Minimalista Baseada em Streaming e "Citations First"

Focada em uma experiência limpa, similar ao ChatGPT ou Perplexity, mas com ênfase total na origem dos dados.[15]

- **Estratégia de UX**: As citações aparecem como pequenos badges numéricos no final de cada frase. Ao passar o mouse, um popover exibe o trecho do documento. No início de cada resposta, uma seção colapsável de "Fontes Consultadas" mostra quais arquivos foram lidos pelo agente.[15]
- **Práticas de Acessibilidade**: Segue o princípio de "Percebível" da WCAG, garantindo que o contraste das bolhas de chat seja de pelo menos 4.5:1 e que os botões de ação tenham áreas de clique de no mínimo 48x48 pixels.[28]

Proposta 3: Fluxo de Trabalho Agentico com Visibilidade de Processo

Esta proposta é ideal para usuários que desejam entender "como" o agente chegou à resposta, expondo a cadeia de pensamento (Chain of Thought).[4]

- **Estratégia de UX**: Inclui componentes de "Status de Geração" que mostram os passos internos: "Pesquisando documentos...", "Refinando consulta...", "Analisando 3 fragmentos...". Isso utiliza animações discretas para manter o usuário engajado durante períodos de processamento local mais longos.[54]
- **Práticas de Acessibilidade**: Utiliza o padrão "Understandable" da WCAG, fornecendo feedbacks claros em caso de erro de rede ou falha de modelo, evitando estados "mortos" na interface.[13]

Implantação Local via Docker

A containerização de todo o ecossistema garante que o projeto de estudos funcione de forma isolada do sistema operacional hospedeiro, facilitando a gestão de dependências complexas como drivers de GPU e binários de LLM.[6]

Arquitetura de Containers (Docker Compose)

O sistema deve ser orquestrado através de um arquivo docker-compose.yml que define a rede interna e os volumes persistentes.[6]

Serviço	Imagem/Origem	Função no Sistema	Recursos Necessários
ollama	ollama/ollama:latest	Motor de inferência para o LLM e para o modelo de embedding.	GPU recomendada (vRAM 8GB+) ou CPU multi-core.[6]
qdrant	qdrant/qdrant:latest	Banco de dados vetorial para armazenamento de documentos.	Persistência em volume local para evitar perda de dados.[10]
backend	python:3.11-slim	API FastAPI, LangChain e lógica do agente.	Acesso à rede interna para comunicar com Ollama e Qdrant.[10]
frontend	node:20-alpine	Interface Next.js.	Exposição da porta 3000 para acesso via navegador.[16]

Considerações de Hardware e Otimização Local

Ao executar modelos de linguagem localmente, a gestão de recursos é o maior desafio técnico.[8]

1. **Suporte a GPU**: Para usuários com GPUs NVIDIA, o container do Ollama deve ser configurado com o nvidia-container-runtime para permitir a aceleração CUDA. Isso reduz o tempo de resposta de minutos para segundos.[8]
2. **Quantização de Modelos**: Recomenda-se o uso de modelos quantizados em 4-bit (ex: llama3:8b-instruct-q4_0). Eles oferecem um equilíbrio ideal entre acurácia e consumo de memória, permitindo que o sistema rode em hardware doméstico com 16GB de RAM total.[6]
3. **Persistência de Dados**: É fundamental definir volumes nomeados no Docker para as pastas de armazenamento do Qdrant (/qdrant/storage) e do Ollama (/root/.ollama). Sem isso, o usuário teria que reindexar todos os documentos e baixar os modelos novamente

toda vez que reiniciasse os containers.[6]

Ambigüidades e Necessidades de Detalhamento

Para garantir que a implementação final atenda exatamente às necessidades do usuário, os seguintes pontos precisam de maior detalhamento:

- **Especificação de Hardware**: A viabilidade de certas propostas de backend (como o uso de múltiplos agentes simultâneos) depende da capacidade de processamento paralela disponível (CPU vs GPU).
- **Volume de Documentos**: Se a base de documentos for excepcionalmente grande (> 10.000 arquivos), a estratégia de banco de dados vetorial pode precisar de otimizações de indexação HNSW específicas.
- **Privacidade Multitenant**: Não ficou claro se o sistema será usado por um único usuário ou se exige isolamento entre diferentes usuários compartilhando a mesma interface local.

Práticas de Engenharia de Software e Clean Code para IA

O desenvolvimento de sistemas baseados em LLM apresenta o desafio da não-deterministicidade. Aplicar práticas de Clean Code ajuda a isolar essa incerteza.[4]

1. **Imutabilidade de Estados**: No LangGraph, o estado do agente deve ser tratado como imutável a cada transição de nó. Isso facilita o "Time-travel debugging", permitindo que o desenvolvedor inspecione exatamente o que o agente sabia em cada passo do raciocínio.[3]
2. **Validação de Schema (Eydantic)**: Todas as saídas do LLM destinadas a chamadas de ferramentas ou lógica do sistema devem ser validadas contra esquemas Pydantic. Caso o LLM gere um JSON inválido, o backend deve capturar o erro e enviar uma mensagem de "auto-correção" para o modelo tentar novamente.[7]
3. **Injeção de Dependência**: O backend deve injetar o cliente de banco de dados e o cliente de LLM nos serviços. Isso permite o uso de "Mocks" em testes unitários, garantindo que a lógica da API possa ser testada sem gastar ciclos de GPU reais.[45]

Este ecossistema técnico, quando devidamente integrado, resulta em uma ferramenta de estudo poderosa, capaz de demonstrar não apenas a eficácia da IA generativa, mas a importância de uma arquitetura de software sólida para sustentar aplicações de inteligência artificial confiáveis e privadas.[7]

Referências citadas

1. What is RAG (Retrieval Augmented Generation)? - IBM, acessado em março 15, 2026, https://www.ibm.com/think/topics/retrieval-augmented-generation
2. Advanced RAG: Architecture, Techniques, Applications and Use Cases and

Development - LeewayHertz, acessado em março 15, 2026, https://www.leewayhertz.com/advanced-rag/
3. LangGraph: Agent Orchestration Framework for Reliable AI Agents - LangChain, acessado em março 15, 2026, https://www.langchain.com/langgraph
4. Mastering-Agentic-Design-Patterns-with-LangGraph/README.md at main - GitHub, acessado em março 15, 2026, https://github.com/MahendraMedapati27/Mastering-Agentic-Design-Patterns-with-LangGraph/blob/main/README.md
5. Beyond the hype: Why RAG remains essential for modern AI | Pinecone, acessado em março 15, 2026, https://www.pinecone.io/learn/rag-2025/
6. GitHub - noorjotk/local-rag-engine: Local RAG app with zero-config Docker setup. FastAPI + Streamlit + Qdrant + Ollama. Just run `docker-compose up, acessado em março 15, 2026, https://github.com/noorjotk/local-rag-engine
7. Implement a simple RAG with Ollama, Qdrant and LangChain | by Filip - Medium, acessado em março 15, 2026, https://medium.com/@filipzupancic123/implement-a-simple-rag-with-ollama-qdrant-and-langchain-cc47beabc290
8. Building Private RAG (Retrieval-Augmented Generation) Systems on Dedicated GPUs: Enterprise Infrastructure - ServerMania, acessado em março 15, 2026, https://www.servermania.com/kb/articles/private-rag-dedicated-gpu-infrastructure
9. LangChain - Qdrant, acessado em março 15, 2026, https://qdrant.tech/documentation/frameworks/langchain/
10. A Dockerized Retrieval-Augmented Generation (RAG) Document Intelligence System built using FastAPI, Ollama, and Qdrant | by Piyali Das | Mar, 2026 | Medium, acessado em março 15, 2026, https://medium.com/@piyalidas.it/a-dockerized-retrieval-augmented-generation-rag-document-intelligence-system-built-using-fastapi-b34625dc6784
11. Enterprise RAG: Use Cases, Common Pitfalls, & Effective Solutions - Fram, acessado em março 15, 2026, https://wearefram.com/blog/enterprise-rag/
12. Retrieval Augmented Generation (RAG) Best Practices for Shipping Real Products - Redwerk, acessado em março 15, 2026, https://redwerk.com/blog/rag-best-practices/
13. Prototyping a RAG System for Digital Humanities: Exploring AI/ML with Indigenous Data, acessado em março 15, 2026, https://kdl.kcl.ac.uk/blog/ireal-rag/
14. Building AI Chat Applications: Streaming, Citations & RAG | Graphlit Developer Guides, acessado em março 15, 2026, https://www.graphlit.com/guides/building-ai-chat-applications
15. React AI Sources - shadcn.io, acessado em março 15, 2026, https://www.shadcn.io/ai/sources
16. Dockerizing a RAG Application with FastAPI, LlamaIndex, Qdrant and Ollama, acessado em março 15, 2026, https://otmaneboughaba.com/posts/dockerize-rag-application/
17. Running Multiple Local Models: Memory Management Strategies - SitePoint, acessado em março 15, 2026,

https://www.sitepoint.com/multiple-local-models-memory-management/
18. Retrieval Augmented Generation (RAG) - Open WebUI, acessado em março 15, 2026, https://docs.openwebui.com/features/chat-conversations/rag/
19. My hands-on experience with Qdrant and Docling (and Ollama) - DEV Community, acessado em março 15, 2026, https://dev.to/aairom/my-hands-on-experience-with-qdrant-and-docling-and-ollama-a09
20. Building a Knowledge Base for RAG Applications - Astera Software, acessado em março 15, 2026, https://www.astera.com/type/blog/building-a-knowledge-base-rag/
21. Implementing RAG using Langchain, Ollama and Qdrant | by Spandan Maity - Medium, acessado em março 15, 2026, https://medium.com/@spandanmaity58/implementing-rag-using-langchain-ollama-and-qdrant-8b7b832fc3da
22. Build a RAG agent with LangChain, acessado em março 15, 2026, https://docs.langchain.com/oss/python/langchain/rag
23. Technical Overview Chat with RAG, acessado em março 15, 2026, https://vrraj.github.io/chat-with-rag/technical-overview.html
24. Production-ready Full-Stack AI Agent Template — FastAPI + Next.js with 5 AI frameworks (PydanticAI, LangChain, LangGraph, CrewAI, DeepAgents), WebSocket streaming, tool approval UI, auth, multi-DB, observability, and 20+ integrations. - GitHub, acessado em março 15, 2026, https://github.com/vstorm-co/full-stack-ai-agent-template
25. acessado em março 15, 2026, https://latenode.com/blog/ai-frameworks-technical-infrastructure/vector-databases-embeddings/best-vector-databases-for-rag-complete-2025-comparison-guide
26. Top 5 Open Source Vector Databases for 2025 (Milvus vs. Qdrant. vs Weaviate vs Faiss. etc.) | by Fendy Feng | Medium, acessado em março 15, 2026, https://medium.com/@fendylike/top-5-open-source-vector-search-engines-a-comprehensive-comparison-guide-for-2025-e10110b47aa3
27. Accessibility - React, acessado em março 15, 2026, https://legacy.reactjs.org/docs/accessibility.html
28. WCAG 2.1 Checklist: Interactive Accessibility Guide, acessado em março 15, 2026, https://www.levelaccess.com/resources/must-have-wcag-2-1-checklist/
29. 10 Essential User Interface Design Best Practices for 2025 | Up North Media, acessado em março 15, 2026, https://upnorthmedia.co/blog/user-interface-design-best-practices
30. CPU vs GPU: Which Wins for Running LLMs Locally? - El Bruno, acessado em março 15, 2026, https://elbruno.com/2025/01/02/cpu-vs-gpu-which-wins-for-running-llms-locally/
31. What's the best vector database for building AI products? | Liveblocks blog, acessado em março 15, 2026, https://liveblocks.io/blog/whats-the-best-vector-database-for-building-ai-products

32. Qdrant integration - Docs by LangChain, acessado em março 15, 2026, https://docs.langchain.com/oss/python/integrations/vectorstores/qdrant
33. Best Vector Databases in 2026: A Complete Comparison Guide - Firecrawl, acessado em março 15, 2026, https://www.firecrawl.dev/blog/best-vector-databases
34. Local RAG tutorial - FastAPI & Ollama & pgvector - Reddit, acessado em março 15, 2026, https://www.reddit.com/r/Rag/comments/1oavnif/local_rag_tutorial_fastapi_ollama_pgvector/
35. Monolithic vs Microservices - Difference Between Software Development Architectures, acessado em março 15, 2026, https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/
36. Advanced RAG Techniques for High-Performance LLM Applications - Neo4j, acessado em março 15, 2026, https://neo4j.com/blog/genai/advanced-rag-techniques/
37. RAG techniques: From naive to advanced - Weights & Biases, acessado em março 15, 2026, https://wandb.ai/site/articles/rag-techniques/
38. Evaluating Advanced RAG Retrievers: A Practical Comparison ..., acessado em março 15, 2026, https://thedataguy.pro/blog/2025/05/evaluating-advanced-rag-retrievers/
39. When Your RAG Chatbot Fails Your Customers: Advanced Retrieval Strategies That Actually Work-Part 6... | by Vineet Chachondia | Medium, acessado em março 15, 2026, https://medium.com/@vineetchachondia/when-your-rag-chatbot-fails-your-customers-advanced-retrieval-strategies-that-actually-work-part-6-c7831f894010
40. RAG Architecture: Complete Guide to Retrieval-Augmented Generation Components, acessado em março 15, 2026, https://latenode.com/blog/ai-frameworks-technical-infrastructure/rag-retrieval-augmented-generation/rag-architecture-complete-guide-to-retrieval-augmented-generation-components
41. Advanced Retrieval Strategies for RAG - Ailog RAG, acessado em março 15, 2026, https://app.ailog.fr/en/blog/guides/retrieval-strategies
42. Building AI Tools That Actually Work: Python, FastAPI & LangChain Made Simple, acessado em março 15, 2026, https://pub.towardsai.net/building-ai-tools-that-actually-work-python-fastapi-langchain-made-simple-d2be283197a8
43. How to Build Agentic AI with LangChain and LangGraph - Codecademy, acessado em março 15, 2026, https://www.codecademy.com/article/agentic-ai-with-langchain-langgraph
44. FastEmbed: Qdrant's Efficient Python Library for Embedding Generation, acessado em março 15, 2026, https://qdrant.tech/articles/fastembed/
45. How Design Patterns Saved My FastAPI Project | by Juan D Khusuma - Medium, acessado em março 15, 2026, https://medium.com/@juan.d.khusuma/how-design-patterns-saved-my-fastapi-p

roject-665a1160c02f
46. Monolithic vs Microservices: Features, Pros & Cons, and Real-World Use Cases, acessado em março 15, 2026, https://hatchworks.com/blog/software-development/monolithic-vs-microservices/
47. Monolith vs Microservices vs Modulith - DZone, acessado em março 15, 2026, https://dzone.com/articles/monolith-vs-modulith-vs-microservices
48. Build a custom RAG agent with LangGraph - Docs by LangChain, acessado em março 15, 2026, https://docs.langchain.com/oss/python/langgraph/agentic-rag
49. Open-sourcing a LangGraph design patterns for building LLM agents : r/SideProject - Reddit, acessado em março 15, 2026, https://www.reddit.com/r/SideProject/comments/1rlm109/opensourcing_a_langgraph_design_patterns_for/
50. 7 Must-Know Agentic AI Design Patterns - MachineLearningMastery.com, acessado em março 15, 2026, https://machinelearningmastery.com/7-must-know-agentic-ai-design-patterns/
51. Top 4 Agentic AI Design Patterns for Architecting AI Systems - Analytics Vidhya, acessado em março 15, 2026, https://www.analyticsvidhya.com/blog/2024/10/agentic-design-patterns/
52. How to Develop AI Agents Using LangGraph: A Practical Guide - freeCodeCamp.org, acessado em março 15, 2026, https://www.freecodecamp.org/news/how-to-develop-ai-agents-using-langgraph-a-practical-guide/
53. Free React & shadcn/ui Templates, acessado em março 15, 2026, https://shadcn.io/template
54. Super 7 Shadcn Chat UI Example for Your Project, acessado em março 15, 2026, https://shadcnstudio.com/blog/shadcn-chat-ui-example
55. Shadcn Studio - Shadcn UI Components, Blocks & Templates, acessado em março 15, 2026, https://shadcnstudio.com/
56. React UI with shadcn/ui + Radix + Tailwind | Vercel Academy, acessado em março 15, 2026, https://vercel.com/academy/shadcn-ui
57. Shadcn UI React Components, acessado em março 15, 2026, https://www.shadcn.io/ui
58. Official Tailwind UI Components & Templates - Tailwind Plus, acessado em março 15, 2026, https://tailwindcss.com/plus
59. Introduction to shadcn/ui: Build Beautiful, Accessible React Components | by Sikandar Dev, acessado em março 15, 2026, https://medium.com/@the.sikandar.dev/introduction-to-shadcn-ui-build-beautiful-accessible-react-components-46dfa3474295
60. 5 Things AI Can't Do, Even in ShadcnUI - DEV Community, acessado em março 15, 2026, https://dev.to/devunionx/5-things-ai-cant-do-even-inshadcnui-6k0
61. Chatbot UI Design: 12 Best Practices for Mobile Apps in 2025 - Widget Chat, acessado em março 15, 2026, https://widget-chat.com/blog/chatbot-ui-design-best-practices
62. The Ultimate Guide to AI Design Patterns for Next-Gen UX - Idea Theorem,

acessado em março 15, 2026, https://ideatheorem.com/insights/blog/the-ultimate-guide-to-ai-design-patterns-for-next-gen-ux
63. Cult UI – Shadcn UI Components, Blocks & Templates, acessado em março 15, 2026, https://www.cult-ui.com/
64. miskibin/chat-components: A customizable, accessible chatGPT like UI components built for React applications using the shadcn/ui design system. - GitHub, acessado em março 15, 2026, https://github.com/miskibin/chat-components
65. Chatbot UI UX Design Best Practices and Examples: Lollypop, acessado em março 15, 2026, https://lollypop.design/blog/2025/january/chatbot-ui-ux-design-best-practices-examples/
66. Web Content Accessibility Guidelines (WCAG) 2.1 - W3C, acessado em março 15, 2026, https://www.w3.org/TR/WCAG21/
67. 16 Chat UI Design Patterns That Work in 2025 - BRICX, acessado em março 15, 2026, https://bricxlabs.com/blogs/message-screen-ui-deisgn
68. Best practices for running a CPU-only RAG chatbot in production? : r/LocalLLaMA - Reddit, acessado em março 15, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1qaf985/best_practices_for_running_a_cpuonly_rag_chatbot/
69. How to build production-ready AI agents with RAG and FastAPI - The New Stack, acessado em março 15, 2026, https://thenewstack.io/how-to-build-production-ready-ai-agents-with-rag-and-fastapi/
70. Compare the Top 7 RAG Frameworks in 2025 - Pathway, acessado em março 15, 2026, https://pathway.com/rag-frameworks
71. RAG Frameworks: Top 5 Picks for Enterprise AI (Nov 2025) - AlphaCorp AI, acessado em março 15, 2026, https://alphacorp.ai/top-5-rag-frameworks-november-2025/